The Language Astro



Astro is the first of five languages designed for a compiler course.

CONTENTS

1 Introduction • 2 Language Description • 3 Standard Library • 4 Formal Syntax • 5 Formal Semantics

1 Introduction

Astro is a fairly trivial programming language with interesting features that make it a great fit for introducing (1) compiler and interpreter writing, and (2) formal language semantics.

This document defines the language Astro.

2 Language Description

2.1 Programs

A **program** is a sequence of one or more statements. There are only two kinds of statements, assignments and print statements. Comments begin with `//` and extend to the end of the line.

```
// A simple program in Astro

radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // another assignment
print(hypot(2.28, 3 - radius) / the_area);    // print statement
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Astro are instances of a **type**. The language has the following types:

- The type `number` of IEEE-754 binary64 values.
- The types `function<`n`>` for all $n \geq 0$, representing functions from n numeric inputs to a single numeric output.

Numbers are first-class values, meaning they can be stored in variables, passed to functions, and returned from functions. Functions cannot: the only thing one can do with a function is call it.

Numbers are written as in JavaScript:

```
2
2.0
55.9
819.999e-15
2E+10
5.89999e2
```

2.3 Assignments

An **assignment** binds an entity to an identifier. Any identifier other than the built-in ones must be assigned to before it can subsequently be used.

```
sister = 5 + 1;              // variable declaration of sister (V + I = VI)
print(sister);               // okay, sister has been assigned
```

```
// print(cousin);          // ERROR: cousin has not been assigned
print(π);                  // Turns out to be okay because π is built-in
```

2.4 Functions

As Astro is a trivial language, you cannot define your own functions. You are limited to the built-in functions `sqrt`, `sin`, `cos`, and `hypot`. If a function is bound to an identifier, you cannot rebind the identifier:

```
print(sqrt(100));          // ok, sqrt are built-in
// sqrt = 5;               // ERROR: function already bound to sqrt
```

Functions can only be called. They cannot be used in a context where a number is expected:

```
// print(sin)              // ERROR
// t = sin;                // ERROR
// strange = sin * 3       // ERROR
```

Functions declared with n parameters must be passed exactly n arguments when called.

2.5 Variables

A **variable** is something that stores a value. Variables come into existence when assigned to for the first time. There is, however, one pre-declared variable, π. Variables can only be used if previously assigned or if built-in.

The variable `π` is read-only and all other variables are writable.

Variables can store numeric values only, not functions.

2.6 Statements

A **statement** is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- *i* = *e* ;

  (Assignment statement) *e* is evaluated, then the value of *e* is copied into *i*. *i* must not already have a function bound to it.

- **print** *e* ;

  (Print statement) Evaluate *e* then writes its evaluation to standard output.

2.7 Expressions

An **expression** produces a numeric. The Astro expressions are:

- A numeric literal.
- A variable occurrence, which must have been previously defined.
- – *e*

  Evaluates *e* and produces the negation of *e*.

- e_1 ** e_2

  The subexpressions are evaluated in any order and $e_1^{e_2}$ is produced.

- e_1 * e_2

  The subexpressions are evaluated in any order and their product is produced.

- e_1 / e_2

  The subexpressions are evaluated in any order and their quotient is produced.

- e_1 % e_2

  The subexpressions are evaluated in any order and the remainder of e_1 divided by e_2 is produced.

- e_1 + e_2

  The subexpressions are evaluated in any order and their sum is produced.

- e_1 – e_2

  The subexpressions are evaluated in any order and their difference is produced.

- $f(e_1, \ldots e_n)$

  (Function call) Evaluates each e_i then calls the function bound to f with these evaluated arguments, in order, and produces the returned value of the function. f must have a function of n parameters bound to it.

3 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program.

- `π`

  Read-only variable whose value is the best approximate value of π.

- `function sqrt(x)`

  Returns the square root of x.

- `function sin(x)`

  Returns the sine of x radians.

- `function cos(x)`

  Returns the cosine of x radians.

- `function hypot(x, y)`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.

4 Formal Syntax

The source of a Astro program is a Unicode string. Here is the syntax given as an Ohm grammar:

astro.ohm

```
Astro {
  Program     = Statement+
```

```
  Statement   = id "=" Exp ";"                          --assignment
              | print Exp ";"                           --print
  Exp         = Exp ("+" | "-") Term                    --binary
              | Term
  Term        = Term ("*" | "/" | "%") Factor           --binary
              | Factor
  Factor      = Primary "**" Factor                     --binary
              | "-" Primary                             --negation
              | Primary
  Primary     = id "(" ListOf<Exp, ","> ")"             --call
              | numeral                                 --num
              | id                                      --id
              | "(" Exp ")"                             --parens

  numeral     = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  print       = "print" ~idchar
  idchar      = letter | digit | "_"
  id          = ~print letter idchar*
  space      += "//" (~"\n" any)*                       --comment
}
```

5 Formal Semantics

The meaning of an Astro program is defined in this section via transition rules in the style of Natural Semantics. It is defined from the following abstract syntax:

n : Nml		
i : Ide		
e : Exp	$=$	$n \mid i \mid -e \mid e+e \mid e-e \mid e*e \mid e/e \mid e\%e \mid e**e \mid \textbf{call}\ i\ e^*$
s : Stm	$=$	$i = e \mid \textbf{print}\ e$
p : Pro	$=$	$\textbf{program}\ s^+$

The meaning of a Astro program at runtime is the list of values it prints. To formally specify this behavior, we also have to define the meanings of statements and expressions. We do this with the help of a **memory**, which maps identifiers to their runtime values, and the **output**, which is the list of values output so far. The type **Value** is defined as:

$$\frac{}{\mathsf{Undef} : \mathsf{Value}} \qquad \frac{x : \mathsf{Real} \quad b : \mathsf{Bool}}{\mathsf{Num}\ x\ b : \mathsf{Value}} \qquad \frac{f : \mathsf{Real}^* \to \mathsf{Real} \quad n : \mathsf{Nat}}{\mathsf{Fun}\ f\ n : \mathsf{Value}}$$

allowing identifiers to be (1) bound to a variable with a mutability flag, (2) bound to a function together with its parameter count (so that the number of arguments can be checked at call time), or (3) not yet defined. The predefined types **Bool**, **Nat**, and **Real** refer to booleans, natural numbers, and IEEE-754 binary64 values, respectively. Each statement is executed in the context of a **state**, which is the current memory together with the output so far. Expressions need only be evaluated in the context of the current memory, as they do not read nor modify the output. The semantic rules are:

$$\frac{}{m \vdash [\![n]\!] \Downarrow n} \qquad \frac{m(i) = \mathsf{Num}\ x\ b}{m \vdash [\![i]\!] \Downarrow x} \qquad \frac{m \vdash e \Downarrow x}{m \vdash [\![- e]\!] \Downarrow -x}$$

$$\frac{op \in \{+, -, *, /, \%, **\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow op(x, y)} \qquad \frac{(m \vdash e_i \Downarrow a_i)_{i=1}^{n} \quad m(i) = \mathsf{Fun}\ f\ n}{m \vdash [\![\mathtt{call}\ i\ e_1, \ldots, e_n]\!] \Downarrow f(a_1, \ldots, a_n)}$$

$$\frac{m \vdash e \Downarrow x \quad m(i) = \mathsf{Undef} \vee m(i) = \mathsf{Num}\ y\ \mathsf{true}}{(m, o) \vdash [\![i = e]\!] \Downarrow (m[i \mapsto \mathsf{Num}\ x\ \mathsf{true}], o)} \qquad \frac{m \vdash e \Downarrow x}{(m, o) \vdash [\![\mathtt{print}\ e]\!] \Downarrow (m, o \cdot x)}$$

$$\frac{((m_{i-1}, o_{i-1}) \vdash s_i \Downarrow (m_i, o_i))_{i=1}^{n}}{\vdash [\![\mathtt{program}\ s_1, \ldots, s_n]\!] \Downarrow o_n}$$

where o_0, the initial output, is defined to be the empty sequence, and the initial memory m_0 is our "standard library" defined as follows:

$$\begin{aligned}
m_0 = (&\lambda i.\ \mathsf{Undef})[\\
&\pi \mapsto \mathsf{Num}\ \pi\ \mathsf{false}][\\
&\mathtt{sqrt} \mapsto \mathsf{Fun}\ (\lambda x.\ \sqrt{x})\ 1][\\
&\mathtt{sin} \mapsto \mathsf{Fun}\ (\lambda x.\ \sin x)\ 1][\\
&\mathtt{cos} \mapsto \mathsf{Fun}\ (\lambda x.\ \cos x)\ 1][\\
&\mathtt{hypot} \mapsto \mathsf{Fun}\ (\lambda (x, y).\ \sqrt{x^2 + y^2})\ 2]
\end{aligned}$$

The Language Bella

Bella is the second of five languages designed for a compiler course.

CONTENTS

1 Introduction • 2 Language Description • 3 Standard Library • 4 Formal Syntax • 5 Formal Semantics

1 Introduction

Bella is a simple programming language with interesting features that make it a great fit for learning (1) compiler and interpreter writing, and (2) formal language semantics.

The language is limited to numbers, assignments, conditionals, loops, and simple functions. Variables can only hold numbers, not functions. Function "bodies" are limited to simple expressions (like Python lambdas), so we don't have to deal with any scoping complications like temporal dead zones. Scoping is as in Python: local variables and parameters are function-scoped, and shadowing is allowed. Bella is a friendly, simple language, relatively easy to implement.

This document defines the language Bella.



2 Language Description

2.1 Programs

A **program** is a sequence of one or more statements.

```
let dozen = 12;                                    // variable declaration
print dozen % 3 ** 1;                              // print statement
function gcd(x, y) = y == 0 ? x : gcd(y, x % y);   // function declaration
while dozen >= 3 || (gcd(1, 10) != 5) {            // while statement
  dozen = dozen - 2.75E+19 ** 1 ** 3;              // assignment statement
}
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Bella are instances of a **type**. The language has the following types:

- The type `number` of IEEE-754 binary64 values.
- The types `function<`n`>` for all $n \geq 0$, representing functions from n numeric inputs to a single numeric output.

Numbers are first-class values, meaning they can be stored in variables, passed to functions, and returned from functions. Functions cannot be: the only thing one can do with a function is call it.

Numbers are written as in JavaScript:

```
2
2.0
55.9
819.999e-15
2E+10
5.89999e2
```

2.3 Declarations

A **declaration** binds an identifier to an entity. There are three kinds of declarations:

- function declarations
- variable declarations

- parameter declarations

Here is a short example showing all three kinds:

```
let sister = 5 + 1;           // variable declaration of sister (V + I = VI)
function triple(x) = x * 3;   // function declaration of triple
                              // ...and parameter declaration of x
```

Each occurrence of an identifier is either a **defining** occurrence or a **using** occurrence. Using occurrences are legal only in the **visible region** of the declaration that declares the identifier. In Bella, **shadowing** of top-level variables by parameters of local functions is permitted, so the visible region of a declaration is the scope minus any "inner" scopes of declarations of identifiers with the same name and may therefore be discontinuous.

- The scope of an identifier declared in a parameter declaration is the body of the innermost function in which the parameter declaration appears.
- The scope of an identifier declared in a variable declaration begins immediately after its initializer and extends to the end of the program.
- The scope of an identifier declared in a function declaration begins immediately after its parameter list and extends to the end of the program.

The identifiers appearing as defining occurrences of variable and function declarations must be mutually exclusive. A function's parameters must be mutually exclusive as well:

```
let x = 3;
// let x = 5;              // ERROR: x already declared
// function x() = 0;       // ERROR: x already declared
// function g(a, a) = 0;   // ERROR: a already declared
function h(x) = 0;         // OK! parameter x shadows global x
// let h = 3;              // ERROR: h already declared (prev line)
function j(k) = π - x;     // OK
let k = 2;                 // OK! Parameter k on prev line not in scope
```

2.4 Functions

As a simple language, Bella does not have any anonymous functions; all functions must be named. The identifier used in the function declaration is read-only variable: you cannot reassign it:

```
function successor(n) = n + 1;
// let successor = 5;            // ERROR: read only
```

Functions can only be called. They cannot be used in a context where a number is expected:

```
function triple (x) = x * 3;
print triple(8);                 // Function call of triple, perfectly fine
// print triple                  // ERROR
// let t = triple;               // ERROR
// let times_nine = triple * 3   // ERROR
```

Functions declared with n parameters must be passed exactly n arguments when called.

2.5 Variables

A **variable** is something that stores a value. Variables come into existence either (1) as regular variables declared in a variable declaration statement (using `let`), or (2) as parameters in a parameter declaration. There is one pre-declared variable, π. Variables can only be used if previously declared.

The variable `π` is read-only and all other variables are writable. Interestingly, because parameters are scoped only to the function body and function bodies are simple expressions, all parameters are effectively read-only.

Variables can store numeric values only, not functions.

2.6 Statements

A **statement** is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- `let` i `=` e `;`

(Variable declaration statement) Declares a new variable with name i. This declaration must be unique within its scope. Evaluates e and assigns this value to the new variable. The new variable is writable.

- `function` f `(` x_1`,...,`x_n `)` `=` e `;`

  (Function declaration) Defines a new function named f with n parameters, that is, its type is `function<`n`>`. The identifier f must be unique within its scope. The parameters must be unique within their scope.

- i `=` e `;`

  (Assignment statement) e is evaluated, then the value of e is copied into i. i must have been previously declared and visible in this scope.

- `print` e `;`

  (Print statement) Evaluates e then prints its value to standard output.

- `while` e b

  (While statement) First, e is evaluated. If e produces 0, the execution of the while statement terminates. Otherwise, body b is executed then the entire while statement is executed again.

2.7 Expressions

An **expression** produces a numeric. The Bella expressions are:

- A numeric literal.
- `true`

  Produces 1.

- `false`

  Produces 0.

- A variable occurrence, which must have been previously declared and visible in this scope.
- `-` e

  Evaluates e and produces the negation of e.

- `!` e

If e evaluates to 0, produces 1, otherwise produces 0.

- e_1 `*` e_2

  The subexpressions are evaluated in any order and their product is produced.

- e_1 `/` e_2

  The subexpressions are evaluated in any order and their quotient is produced.

- e_1 `%` e_2

  The subexpressions are evaluated in any order and the remainder of e_1 divided by e_2 is produced.

- e_1 `+` e_2

  The subexpressions are evaluated in any order and their sum is produced.

- e_1 `-` e_2

  The subexpressions are evaluated in any order and their difference is produced.

- e `?` e_1 `:` e_2

  Evaluates e and if non-zero, evaluates and produces e_1. Otherwise evaluates and produces e_2.

- e_1 `<` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is less than the value of e_2.

- e_1 `<=` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is less or equal to the value of e_2.

- e_1 `==` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is equal to the value of e_2.

- e_1 `!=` e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is not equal to the value of e_2.

- e_1 >= e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is greater or equal to the value of e_2.

- e_1 > e_2

  Evaluates the subexpressions in any order, then produces 1 or 0, respectively, as to whether the value of e_1 is greater than the value of e_2.

- e_1 && e_2

  First e_1 is evaluated. If it evaluates to 0, the entire expression immediately produces 0 (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 || e_2

  First e_1 is evaluated. If it evaluates to a non-zero value, the entire expression immediately produces this value (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

3 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program.

- π

  Read-only variable whose value is the best approximate value of π.

- `function sqrt(x)`

  Returns the square root of x.

- `function sin(x)`

  Returns the sine of x radians.

- `function cos(x)`

  Returns the cosine of x radians.

- `function exp(x)`

  Returns e^x.

- `function ln(x)`

  Returns the natural log of x.

- `function hypot(x, y)`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.

4 Formal Syntax

The source of a Bella program is a Unicode string. Here is the syntax given as an Ohm grammar:

bella.ohm

```
Bella {
  Program   = Statement+
  Statement = let id "=" Exp ";"                          -- vardec
            | function id Params "=" Exp ";"              -- fundec
            | Exp7_id "=" Exp ";"                         -- assign
            | print Exp ";"                               -- print
            | while Exp Block                             -- while
  Params    = "(" ListOf<id, ","> ")"
  Block     = "{" Statement* "}"

  Exp       = ("-" | "!") Exp7                            -- unary
            | Exp1 "?" Exp1 ":" Exp                       -- ternary
            | Exp1
  Exp1      = Exp1 "||" Exp2                              -- binary
            | Exp2
  Exp2      = Exp2 "&&" Exp3                              -- binary
            | Exp3
  Exp3      = Exp4 ("<="|"<"|"=="|"!="|">="|">") Exp4     -- binary
            | Exp4
  Exp4      = Exp4 ("+" | "-") Exp5                       -- binary
            | Exp5
```

```
  Exp5      = Exp5 ("*" | "/" | "%") Exp6                -- binary
            | Exp6
  Exp6      = Exp7 "**" Exp6                             -- binary
            | Exp7
  Exp7      = num
            | true
            | false
            | id "(" ListOf<Exp, ","> ")"                -- call
            | id                                         -- id
            | "(" Exp ")"                                -- parens

  let       = "let" ~idchar
  function  = "function" ~idchar
  while     = "while" ~idchar
  true      = "true" ~idchar
  false     = "false" ~idchar
  print     = "print" ~idchar
  keyword   = let | function | while | true | false
  num       = digit+ ("." digit+)? (("E" | "e") ("+" | "-")? digit+)?
  id        = ~keyword letter idchar*
  idchar    = letter | digit | "_"
  space    += "//" (~"\n" any)*                          -- comment
}
```

5 Formal Semantics

The meaning of a Bella program is defined in this section via transition rules in the style of Natural Semantics. It is defined from the following abstract syntax:

n: Numeral
i: Identifier
e: Expression $= n \mid i \mid \texttt{true} \mid \texttt{false} \mid \mathit{unaryop}\ e \mid e_1\ \mathit{binop}\ e_2 \mid \texttt{call}\ i\ e^* \mid e\ \texttt{?}\ e_1\ \texttt{:}\ e_2$
s: Statement $= \texttt{let}\ i = e \mid \texttt{func}\ i\ i^* = e \mid i = e \mid \texttt{print}\ e \mid \texttt{while}\ e\ b$
b: Block $= \texttt{block}\ s^*$
p: Program $= \texttt{program}\ b$

The unary operators are `-` and `!`. The binary operators are `+`, `-`, `*`, `/`, `%`, `**`, `<`, `<=`, `==`, `!=`, `>=`, `>`, `&&`, and `||`.

The meaning of a Astro program at runtime is the list of values it prints. To formally specify this behavior, we also have to define the meanings of statements and expressions. We do this with the help of a **memory**, which maps identifiers to their runtime values, and the **output**, which is the list of values output so far. The type **Value** is defined as:

$$\frac{}{\textsf{Undef}\colon \textsf{Value}} \qquad \frac{x\colon \textsf{Real} \quad b\colon \textsf{Bool}}{\textsf{Num}\ x\ b\colon \textsf{Value}} \qquad \frac{p\colon \textsf{Ide}^* \quad e\colon \textsf{Exp}}{\textsf{Fun}\ p\ e\colon \textsf{Value}} \qquad \frac{f\colon \textsf{Real}^* \to \textsf{Real} \quad n\colon \textsf{Nat}}{\textsf{StdFun}\ f\ n\colon \textsf{Value}}$$

allowing identifiers to be (1) bound to a variable with a mutability flag, (2) bound to a (built-in, or "standard") function together with its parameter count (so that the number of arguments can be checked at call time), (3) a user-defined function with a parameter list and a body expression, or (4) not yet defined. The predefined types **Bool**, **Nat**, and **Real** refer to booleans, natural numbers, and IEEE-754 binary64 values, respectively. Each statement is executed in the context of a **state**, which is the current memory together with the output so far. Expressions need only be evaluated in the context of the current memory, as they do not read nor modify the output. The semantic rules are:

$$\frac{}{m \vdash [\![n]\!] \Downarrow n} \qquad \frac{}{m \vdash [\![\texttt{true}]\!] \Downarrow 1} \qquad \frac{}{m \vdash [\![\texttt{false}]\!] \Downarrow 0}$$

$$\frac{m(i) = \textsf{Num}\ x\ b}{m \vdash [\![i]\!] \Downarrow x} \qquad \frac{m \vdash e \Downarrow x}{m \vdash [\![- e]\!] \Downarrow -x} \qquad \frac{m \vdash e \Downarrow x \quad x \neq 0}{m \vdash [\![\texttt{!}\ e]\!] \Downarrow 0}$$

$$\frac{m \vdash e \Downarrow 0}{m \vdash [\![\texttt{!}\ e]\!] \Downarrow 1} \qquad \frac{op \in \{+, -, *, /, \%, **\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow op(x, y)}$$

$$\frac{op \in \{<, <=, ==, \texttt{!}=, >=, >\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y \quad op(x, y)}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow 1} \qquad \frac{op \in \{<, <=, ==, \texttt{!}=, >=, >\} \quad m \vdash e_1 \Downarrow x \quad m \vdash e_2 \Downarrow y \quad \neg op(x, y)}{m \vdash [\![e_1\ op\ e_2]\!] \Downarrow 0}$$

$$\frac{m \vdash e_1 \Downarrow 0}{m \vdash [\![e_1\ \&\&\ e_2]\!] \Downarrow 0} \qquad \frac{m \vdash e_1 \Downarrow x \quad x \neq 0 \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ \&\&\ e_2]\!] \Downarrow y}$$

$$\frac{m \vdash e_1 \Downarrow x \quad x \neq 0}{m \vdash [\![e_1\ ||\ e_2]\!] \Downarrow x} \qquad \frac{m \vdash e_1 \Downarrow 0 \quad m \vdash e_2 \Downarrow y}{m \vdash [\![e_1\ ||\ e_2]\!] \Downarrow y}$$

$$\frac{m \vdash e \Downarrow x \quad x \neq 0 \quad m \vdash e_1 \Downarrow y}{m \vdash [\![e \texttt{ ? } e_1 \texttt{ : } e_2]\!] \Downarrow y} \qquad \frac{m \vdash e \Downarrow 0 \quad m \vdash e_2 \Downarrow z}{m \vdash [\![e \texttt{ ? } e_1 \texttt{ : } e_2]\!] \Downarrow z}$$

$$\frac{(m \vdash e_i \Downarrow a_i)_{i=1}^n \quad m(i) = \mathsf{Fun}\ (p_1, \ldots, p_n)\ e' \quad m[p_i \mapsto a_i]_{i=1}^n \vdash e' \Downarrow x}{m \vdash [\![\texttt{call } i\ e_1, \ldots, e_n]\!] \Downarrow x}$$

$$\frac{(m \vdash e_i \Downarrow a_i)_{i=1}^n \quad m(i) = \mathsf{StdFun}\ f\ n}{m \vdash [\![\texttt{call } i\ e_1, \ldots, e_n]\!] \Downarrow f(a_1, \ldots, a_n)}$$

$$\frac{m \vdash e \Downarrow x \quad m(i) = \mathsf{Undef}}{(m, o) \vdash [\![\texttt{let } i = e]\!] \Downarrow (m[i \mapsto \mathsf{Num}\ x\ \mathsf{true}], o)}$$

$$\frac{m(i) = \mathsf{Undef}}{(m, o) \vdash [\![\texttt{fun } i\ (p_1, \ldots, p_n) = e]\!] \Downarrow (m[i \mapsto \mathsf{Fun}\ (p_1, \ldots, p_n)\ e], o)}$$

$$\frac{e, m \Downarrow x \quad m(i) = (\mathsf{Num}\ y\ \mathsf{true})}{(m, o) \vdash [\![i = e]\!] \Downarrow (m[i \mapsto (\mathsf{Num}\ x\ \mathsf{true})], o)} \qquad \frac{m \vdash e \Downarrow x}{(m, o) \vdash [\![\texttt{print } e]\!] \Downarrow (m, o \cdot x)}$$

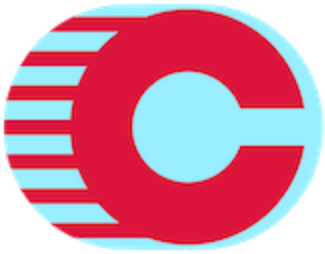
$$\frac{m \vdash e \Downarrow 0}{(m, o) \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m, o)} \qquad \frac{\begin{array}{c} m \vdash e \Downarrow x \quad x \neq 0 \quad (m, o) \vdash b \Downarrow (m', o') \\ (m', o') \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m'', o'') \end{array}}{(m, o) \vdash [\![\texttt{while } e\ b]\!] \Downarrow (m'', o'')}$$

$$\frac{((m_i, o_i) \vdash s_i \Downarrow (m_{i+1}, o_{i+1}))_{i=1}^n}{(m_1, o_1) \vdash [\![\texttt{block } s_1, \ldots, s_n]\!] \Downarrow o_{n+1}} \qquad \frac{(m_0, o_0) \vdash b \Downarrow (m, o)}{[\![\texttt{program } b]\!] \Downarrow o}$$

where o_0, the initial output, is defined to be the empty sequence, and the initial memory m_0 is our "standard library" defined as follows:

$$\begin{aligned}
m_0 = (&\lambda i.\ \mathsf{Undef})[\\
&\pi \mapsto \mathsf{Num}\ \pi\ \mathsf{false}][\\
&\texttt{sqrt} \mapsto \mathsf{Fun}\ (\lambda x.\ \sqrt{x})\ 1][\\
&\texttt{sin} \mapsto \mathsf{Fun}\ (\lambda x.\ \sin x)\ 1][\\
&\texttt{cos} \mapsto \mathsf{Fun}\ (\lambda x.\ \cos x)\ 1][\\
&\texttt{exp} \mapsto \mathsf{Fun}\ (\lambda x.\ e^x)\ 1][\\
&\texttt{ln} \mapsto \mathsf{Fun}\ (\lambda x.\ \ln x)\ 1][\\
&\texttt{hypot} \mapsto \mathsf{Fun}\ (\lambda (x, y).\ \sqrt{x^2 + y^2})\ 2]
\end{aligned}$$

The Language Carlos



The Carlos language was created sometime in the 1990s for a compiler construction class and has evolved a bit since. It is meant to be small, but not trivial, with features interesting enough to make writing a compiler an educational experience, but extremely clean so writing a compiler does not require handling difficult-to-implement tricky cases.

CONTENTS

1 Introduction • 2 Semantics • 3 Syntax • 4 Standard Library

1 Introduction

Carlos is a statically-typed, block-structured programming language with first-class functions. It has no explicit pointers, so assumes the existence of a garbage collector. While several types do have reference semantics, there is no billion-dollar mistake, since the null value is not a member of all reference types, but only of optional types. The language is rather small, with arrays, simple structs, and functions.

```
const languageName = "Carlos";

function greeting(): string {
  return random(["Welcome", "こんにちは", "Bienvenido"]);
}

print("👋👋👋");
repeat 5 {
```

```
  print(greeting() + " " + languageName);
}
```

This document defines the language Carlos.

2 Semantics

2.1 Programs

A **program** is a sequence of one or more statements.

```
// This program prints "hello, world" to standard output.

let greeting = "hello";             // variable declaration
function place(): string {          // function declaration
  return "world";
}
print(greeting + ", " + place());   // function call statement
```

Apologies for the semicolons, but they do make the language somewhat easier to parse.

2.2 Values and Types

All values in Carlos are instances of a type. The language has the following types:

- the type `boolean` consisting entirely of the values `true` and `false`.
- the type `int` of (unbounded) integers.
- the type `float` of IEEE-754 binary64 values.
- the type `string` of character strings.
- the type `void` which has no instances at all.
- the type `any` which is the union of all types.
- *array types*, written `[T]`, whose instances are zero-based integer-indexed sequences of values of type `T`.

- *structure types*, whose instances are ordered sequence of named *fields*.
- *optional types*, written `T?`, representing "boxes" that either contain a value of type `T` or don't contain anything.
- *function types*, written `(T1,T2,...,Tn)->T0`, consisting of all functions mapping (ordered) parameters of types `T1` through `Tn` to a value of type `T0`.

Booleans, integers, floats, and strings have literal forms:

```
true                                // has type boolean
false                               // has type boolean
2                                   // has type int
8912313125712589125870719 2         // has type int
2.0                                 // has type float
55.9                                // has type float
819.999E-15                         // has type float
"¡hola!"                            // has type string
"π™\t\n \" \' \u{1f4a9}"            // has type string
"I'm 💀"                            // has type string
```

The values in an array expression must all have exactly the same type; it is a compile-time error if they do not. Empty arrays *must* mention the intended type of the elements:

```
[true, true, false]                 // has type [boolean]
[3, 0, 5, 99 ** 250]                // has type [int]
[[1, 4], [2, 3]]                    // has type [[int]]
// [3, 0, 2.0]                      // ERROR Cannot mix ints and floats
// [3, "3"]                         // ERROR: Cannot mix ints and strings
[string]()                          // empty array of type [string]
// []                               // Syntax error, cannot infer a type
```

For optionals, use the `some` operator to create an optional with a present value. The operator `no` makes an empty optional on the given type.

```
some 5                              // has type int?
some ["red", "green", "blue"]       // has type [string]?
no boolean                          // has type boolean?
```

There are no literals for functions; they have to be named:

```
function sameMagnitude(x: int, y: int): boolean {
  return x == y || x == -y;
}

function printSecond(messages: [string]) {
  print(messages[1]);
}

function twice(f: (float)->float, x: float): float {
  return f(f(x));
}

sameMagnitude                       // has type (int, int)->boolean
printSecond                         // has type ([string])->void
twice                               // has type ((float)->float, float)->float
```

There are no type expressions for struct types; structure types have to be declared and referred to only by name. Instances of struct types are created using the struct name as a constructor:

```
struct Vector {i: float j: float}   // Declares new type Vector

let v = Vector(2.0, -5.0 * π);      // Structs created by initializing with type

// Only way to refer to this type is via its name
function magnitude(v: Vector): float {
  return hypot(v.i, v.y);
}                                   // type of magnitude is (Vector)->float
```

In Carlos, types are not first-class values. They can be named by identifiers, and you will see both type expressions and type names appear in the source code, but only in parameter declarations, the return type portion of a function declaration, in empty array expressions, with the `no` operator, and in casts. But you cannot assign them to variables or pass them to functions or return them from functions.

```
// let x = int                      // ERROR, int is not a legal expression value
// print(string)                    // ERROR, string is not a legal expression va
```

```
let supervisor = no string          // OK, empty optional of type string?
let scores = [float]()              // OK, just no scores yet
```

2.3 Declarations

A **declaration** binds an identifier to an entity. There are six kinds of declarations:

- type declarations
- function declarations
- variable declarations
- parameter declarations
- iterator declarations
- field declarations

Here is a short example showing all six kinds:

```
let friends = ["EY", "MA", "AA"];   // variable decl: friends
struct Vector {                     // type decl: Vector
  i: float                          // field decl: i
  j: float
}
for friend in friends {             // iterator decl: friend
  print(friend);
}
function triple(x: int): int {      // function decl: triple, parameter decl: x
  return x * 3;
}
```

Each occurrence of an identifier is either a **defining** occurrence or a **using** occurrence. Using occurrences are legal only in the **visible region** of the declaration that declares the identifier. In Carlos, **shadowing** is never permitted, so the visible region of a declaration coincides exactly with the declaration's **scope**. (This is different from most other static languages in which the visible region is the scope minus any "inner" scopes of declarations of identifiers with the same name and may therefore be discontinuous).

- The scope of an identifier declared in a variable, type, or function declaration begins with the declaration itself and extends to the end of the innermost block containing the declaration, or to the end of the program if the declaration does not appear in a block.

- The scope of an identifier declared in a field declaration is the same as the type in which the declaration appears.
- The scope of an identifier declared in a parameter declaration is the body of the innermost function in which the parameter declaration appears.
- The scope of an identifier declared as an iterator is the body of the loop statement in which it appears.

Example. In the following script:

```
function g(a: [int]) {}                          // line 1
let c = g([1, 2, 5]);                            // line 2
let y = c + 2;                                   // line 3
print(c);                                        // line 4
function f(string s) {                           // line 5
  // Note: let y = 0 would be illegal here       // line 6
  struct Point { x: int y: int }                 // line 7
  function q(): Point? { return no Point }       // line 8
  c = 50;                                        // line 9
}                                                // line 10
f("Can't see point or s here")                   // line 11
```

we have the following scopes:

Identifier	declared on line	has scope
g	1	1-11
a	1	body of g
c	2	2-11
y	3	3-11
f	5	5-11
s	5	5-10
Point	7	7-10
q	8	8-10

All declarations in a scope must declare distinct identifiers. Also notice that no shadowing rule only applies to a single path of nested scopes from the top-level scope. Identifiers in “sibling scopes” do

not shadow each other. In addition, struct fields are not part of any path, so there are no restrictions on their names:

```
function test(x: int, y: boolean) {
  const z = 0;
  // let x = "hello";          // ERROR: x is already a parameter
  if z > 1.0 {
    // let x = 2;              // ERROR: shadowing not allowed
    const a = 5;               // OK: new var, local to the if-block
  }
  struct Point {
    x: float                   // OK: fields not part of scope chain
    y: float                   // ...so they don't actually "shadow"
  }
  for i in 0 ..< 10 {
    // let i = 3;              // ERROR: in same scope as iterator i
    while false {
      // let i = 1;           // ERROR: no shadowing
      let a = false;           // OK: does not conflict with the other a
    }
  }                            // No var a is in scope out here though
}
```

Each variable, parameter, or field is constrained to hold values from a certain type. The type constraint for a variable is given in its initializing expression:

```
let gameOver = false;          // gameOver now constrained to booleans
gameOver = true;               // OK: true is boolean
// gameOver = 5.5;             // ERROR: gameOver can only take booleans.
```

Parameters and return types do not have initializers, so Carlos requires their types must be specified explicitly (while in principle, some complex type inference could be done, Carlos wants to be a smaller language that is easier to implement and restricts its type inference to very local situations):

```
function repeated(s: string; n: int): string {
  let result = "";
  repeat n {
    result = result + s;
```

```
  }
  return result;
}
```

2.4 Functions

As a simple language, Carlos does not have any anonymous functions; all functions must be named. The identifier used in the function declaration is essentially a read-only variable: you can assign it to other variables but you cannot reassign it:

```
function next(n: int): int {
  return n + 1;
}

let successor = next;                // OK
print(successor(2));                 // prints 3

// let next = successor;             // ERROR: next is read only
```

Functions are first-class objects, too. They can be assigned and passed as parameters and returned from functions:

```
function triple (x: int): int { return x * 3 }

function twice(f: (int)->int, x: int) { return f(f(x)) }

print(twice(triple, 8));             // prints "72"
```

2.5 Structs

It is possible to write any array, optional, or function type as a type expression from its constituent types; however there is no expression for a structure type. Structure types are defined in a `struct` declaration and bound to an identifier. The identifier is the only way to refer the type.

```
// We can use type expressions for function and array types
function applyToSecondInt(f: (int)->int, a: [int]): int {
  return f(a[1]);
}

struct Vector {i: float j: float}

// But for struct types we have to use a type name!
function magnitude(v: Vector): float {
  return hypot(v.i, v.y);
}
```

The fields within a struct type can be any identifier must be mutually distinct:

```
// struct S {x: int x: int}         // ERROR: duplicate field x
```

No field of a struct is allowed to have the type of the struct itself, as that would lead to an infinitely-sized structure. However, a struct may contain optionals, arrays, or functions that use the type itself:

```
struct S {
  a: S?           // OK: might be empty
  b: [S]          // OK: might be empty
  c: (S) -> S     // OK: function objects are not stored inline
  // d: S         // ERROR: struct would be infinite in size
}
```

2.7 Variables

A variable is something that stores a value. All variables are type-constrained. Variables are either writable (if declared with `let`) or not writable (if declared with `const`).

```
let turnsRemaining = 2;
const adjustment = 0.00137;

turnsRemaining = 3;                 // OK to assign, variable is writable
```

```
turnsRemaining--;                    // OK, ++ and -- are updates
// adjustment = 2.2E-7;              // ERROR: variable is read-only (const)
```

Variables directly store values of booleans, numbers, chars, and strings; however, they can only contain *references* to arrays, structs, functions, and optionals. This means that aliasing of an object can occur:

```
let a = [1, 2, 3]
let b = a;                           // same object pointed to by a and b
a[2] = 1000;                         // updates the object
print(b);                            // prints [1, 2, 1000]
```

The kinds of variables are:

- i

  Simple variable. Identifier i must be bound to a visible variable, parameter, iterator, or function declaration. (This means stand-alone fields are *not* variables, nor are identifiers associated with types.) The variable's type is the type from the declaration, and it is writable iff its declaration used `let`.

- v `.` f

  Member variable. v must be a variable of a struct type. f must be an identifier declared as a field of v's type. The resulting variable refers to the f-field of the object referred to by v, is writable iff v is writable, and its type is the type associated with the field f.

- v `[` e `]`

  Subscripted variable. v must be a variable of an array type, and e must have type `int`. The result is the array component at (zero-based) index e, is writable iff v is writable, and its type is the base type of the array type. If during execution e evaluates to a value less than zero or greater than or equal to the length of v, the program dies.

- v `(` $e_1 \ldots e_n$ `)`

  Call expression. v must either (1) have type $(T_1, \ldots T_n) \rightarrow T_0$ where each e_i has type T_i, or (2) be an identifier bound to a structure type. In the former case, the arguments are evaluated in any order and the function is called with the arguments copied to the parameters, with the result being the return value of the function and its type is the return type of the function type. In the latter case the variable is a newly created struct instance of the given type; the arguments are evaluated and assigned to the fields of the new object. In both cases, the resulting variable is **not** writable.

- *v*`?.`*f*

  Optional Member variable. *v* must be a variable of a optional type wrapping a struct type. *f* must be an identifier declared as a field of *v*'s wrapped type. The resulting variable refers to the *f*-field of the object referred to by *v* if something is wrapped or an empty optional otherwise, is writable iff *v* is writable, and its type is the optional type wrapping the type associated with the field *f*.



```
struct Point {
  x: float
  y: float
}

let p = some Point(3, 5)
let q = no Point

print(p?.x)             // type is int?, value is (some 5)
print(q?.x)             // type is int?, value is (no int)
```

- *v*`?[`*e*`]`

  Optional subscript variable. *v* must be a variable of a optional type wrapping an array type. *e* must have type `int`. The result is the element at (zero-based) index *e* if an array is wrapped or an empty optional otherwise, is writable iff *v* is writable, and its type is the base type of the wrapped array's type. If during execution *e* evaluates to a value less than zero or greater than or equal to the length of *v*, the program dies.



```
let p = some [3, 5, 8]
let q = no [int]

print(p?[1])            // type is int?, value is (some 5)
print(q?[1])            // type is int?, value is (no int)
```

Variables are given values (1) in a variable declaration, (2) in an assignment statement, or (3) if a parameter, during a function call where arguments are assigned to parameters. In all cases, the type *S* of the value being assigned (the source) is checked to see if it is assignable to the type constraint *T* on the variable (the target). The assignment is valid if and only if *S* is **assignable** to *T*, which is the case if and only if:

- *S* is *T*.
- *T* is `any`.

- S is $(S_1, \ldots, S_n) \to S_0$ and T is $(T_1, \ldots, T_n) \to T_0$ and S_0 is assignable to T_0 and each T_i is assignable to S_i.

That is, Carlos covariant in return types, contravariant in parameters, and invariant elsewhere.

2.7 Statements

A statement is code that is executed solely for its side effect; it produces no value. The kinds of statements are:

- `let` i `=` e `;`

  `const` i `=` e `;`

  Variable declaration statement. Declares a new variable with name i. This declaration must be unique within its scope. Evaluates e and assigns this value to the new variable. The new variable will be constrained to be assigned only variables of e's type. The variable is writable iff declared with `let`.

- `struct` S `{` $x_1 : T_1 \ldots x_n : T_n$ `}`

  Struct type declaration. Defines a new struct type named S with the given fields. The identifier S must be unique within its scope, and the fields $x_1 \ldots x_n$ must be mutually distinct. None of the T_i's are allowed to be S itself.

- `function` f `(` $x_1 : T_1 \ldots x_n : T_n$ `)` `:` T_0 b

  Function declaration. Defines a new function named f with type $(T_1, \ldots, T_n) \to T_0$. The identifier f must be unique within its scope. The parameters must be unique within their scope.

- v `=` e `;`

  Assignment statement. e must have the same type that v is constrained to. v is determined and then e is evaluated, then the value of e is copied into v.

- v`++;`

  Increment statement. v must have type `int`. Increments v.

- v`--;`

  Decrement statement. v must have type `int`. Decrements v.

- f`(` $e_1, \ldots, e_n$ `);`

Call statement. f must have type $(T_1, \ldots T_n) \rightarrow$ `void` where each e_i has type T_i. The arguments are evaluated in any order and the function f is called with the arguments copied to the parameters.

- `break;`

  Break statement. This statement may only appear within a loop (`for` , `while` , or `repeat`) statement that is properly within the same function as the break statement. The break statement terminates the execution of the innermost enclosing loop.

- `return;`

  Short Return statement. Causes an immediate return from the innermost enclosing function, whose type must have a return type of `void` .

- `return` e `;`

  Return statement. Evaluates e then causes the innermost enclosing function to immediately return the value of e. e must have the type of the function's type's return type.

- `if` e b

  Short If statement. e must have type `boolean` . If e evaluates to true, b is executed, otherwise nothing else happens.

- `if` e b `else` s

  If statement. e must have type `boolean` . If e evaluates to true, b is executed, otherwise s is executed.

- `while` e b

  While statement. e must have type `boolean` . First e is evaluated. If e produces false the execution of the while statement terminates. If e produces true, b is executed then the entire while-statement is executed again.

- `repeat` e b

  Repeat statement. e is evaluated and must have type `int` . Body b is executed the number of times equal to the value of e. If this value is negative the loop executes zero times.

- `for` i `in` r b

  For-each statement. r must have an array type or string type. r is evaluated and then b is executed repeatedly with i bound to successive elements from r if r is an array, and for successive code points from r if r is a string.

- `for` i `in` e_1 `...` e_2 b

  Through-range iteration statement. e_1 and e_2 must have type `int`. They are first evaluated in any order and then b is executed repeatedly with i bound to successive elements in the range, which goes from e_1 to e_2 inclusive.

- `for` i `in` e_1 `..<` e_2 b

  Upto-range iteration statement. e_1 and e_2 must have type `int`. They are first evaluated in any order and then b is executed repeatedly with i bound to successive elements in the range, which goes from e_1 to e_2 exclusive.

2.8 Expressions

An expression evaluates to a value or crashes the program. Carlos is a statically-typed language, meaning that the type of every expression can be inferred by looking at the program text; there is no need to run the program to determining an expression's type.

The Carlos expressions are:

- An integer literal, which has type `int`.
- A floating point literal, which has type `float`.
- A string literal, which has type `string`.

  String literals are made from a sequence of character specifications enclosed in quotation marks (U+0022). A character specification is either a non-control character, an extended grapheme cluster, or an escape sequence. An escape sequence is one of:

Escape	**Description**
`\n`	newline (U+000A)
`\t`	tab (U+0009)
`\u{` *xxxxxx* `}`	where *xxxxxx* is a one to six character hexadecimal digit sequence standing for a character with a given code point.
`\"`	the quotation mark character (U+0022)
`\'`	the apostrophe character (U+0027)
`\\`	the backslash character (U+005C)

- `true`

  The literal of type `boolean` denoting truth.

- `false`

  The literal of type `boolean` denoting falsity.

- v

  v is a variable. The type of this expression is the type of the variable v, and the value of this expression is the current value stored in v.

- `[` $e_1, \ldots, e_n$ `]`

  Array object construction. Each e_i must have exactly the same type T (there is no allowance for subtyping or anything like that). This expression refers to a newly constructed array of n items consisting of the values of each subexpression, respectively, and has type $[T]$.

- `[` T `]()`

  Empty array. A newly constructed empty array of type $[T]$.

- `some` e

  An optional that wraps e.

- `no` T

  The empty optional of type $T?$.

- `#` e

  Here e must be an expression of some array type. Produces the length of the array. The type of this expression is `int`.

- `random` e

  Here e must be an expression of some array type. Produces a random element from the array.

- `(` e `)`

  Evaluates e and produces its value.

- `-` e

  e must have type `int` or `float`. Evaluates e and produces the negation of e.

- `~` e

e must have type `int`. Evaluates *e* and produces the bitwise complement of *e*.

- `! e`

  e must have type boolean. If *e* evaluates to true, the entire expression produces **false**, otherwise it produces **true**.

- e_1 `*` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their product is produced.

- e_1 `/` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their quotient is produced. The quotient will have the same type as the operands, so division of integers produces the floor division.

- e_1 `%` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an integer which is the remainder of e_1 divided by e_2.

- e_1 `+` e_2

  Both subexpressions must have type `int` or both must have type `float` or both must have type `string`. The subexpressions are evaluated in any order and their sum (for numbers) or concatenation (for strings) is produced.

- e_1 `-` e_2

  Both subexpressions must have type `int` or both must have type `float`. The subexpressions are evaluated in any order and their difference is produced.

- e_1 `<<` e_2

  Each e_i must have type `int`. Produces the value of e_1 shifted left e_2 positions.

- e_1 `>>` e_2

  Each e_i must have type `int`. Produces the value of e_1 arithmetically shifted right e_2 positions.

- e_1 `<=` e_2

Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is less than or equal to the value of e_2.

- e_1 `<` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is less than the value of e_2.

- e_1 `==` e_2

  e_1 and e_2 must have the same type. The subexpressions are evaluated in any order, and the entire expression produces whether these values are equal, defined as follows. Ints, floats, and booleans are equal if they have the same value. Strings are the same if their byte sequences are equal (they are *not* normalized prior to comparison. Arrays, structs, and functions are equal iff the two expressions reference the exact same object. Two optionals are equal iff they are both empty optionals or their wrapped values are equal.

- e_1 `!=` e_2

  Equivalent to `!(e1==e2)`.

- e_1 `>` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is greater than the value of e_2.

- e_1 `>=` e_2

  Each subexpression must have type `int`, or must both have type `float`, or must both have type `string`. Both expressions are evaluated, in any order, and the entire expression produces whether the value of e_1 is greater than or equal to the value of e_2.

- e_1 `&` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise AND of e_1 and e_2.

- e_1 `^` e_2

  Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise XOR of e_1 and e_2.

- e_1 `|` e_2

Each subexpression must have type `int`. Both expressions are evaluated, in any order, and the entire expression produces an int which is the bitwise (inclusive) OR of e_1 and e_2.

- e_1 `&&` e_2

  Each subexpression must have type `boolean`. First e_1 is evaluated. If it evaluates to false, the entire expression immediately produces false (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 `||` e_2

  Each subexpression must have type `boolean`. First e_1 is evaluated. If it evaluates to true, the entire expression immediately produces true (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

- e_1 `??` e_2

  e_1 must have type T`?` for some type T and e_2 must have type T. First e_1 is evaluated. If it wraps a value, the entire expression immediately produces the wrapped value (without evaluating e_2). Otherwise e_2 is evaluated and the entire expression produces the value of e_2.

3 Syntax

The source of a Carlos program is a Unicode string. Here is the grammar in Ohm notation:

carlos.ohm

```
Carlos {
  Program     = Statement+

  Statement   = VarDecl
              | TypeDecl
              | FunDecl
              | Exp9 ("++" | "--") ";"                    --bump
              | Exp9 "=" Exp ";"                          --assign
              | Exp9_call ";"                             --call
              | break ";"                                 --break
              | return Exp ";"                            --return
              | return ";"                                --shortreturn
```

```
            | IfStmt
            | LoopStmt

VarDecl     = (let | const) id "=" Exp ";"
TypeDecl    = struct id "{" Field* "}"
Field       = id ":" Type
FunDecl     = function id Params (":" Type)? Block
Params      = "(" ListOf<Param, ","> ")"
Param       = id ":" Type

Type        = Type "?"                                  --optional
            | "[" Type "]"                              --array
            | "(" ListOf<Type, ","> ")" "->" Type       --function
            | id                                        --id

IfStmt      = if Exp Block else Block                   --long
            | if Exp Block else IfStmt                  --elsif
            | if Exp Block                              --short
LoopStmt    = while Exp Block                           --while
            | repeat Exp Block                          --repeat
            | for id in Exp ("..." | "..<") Exp Block   --range
            | for id in Exp Block                       --collection
Block       = "{" Statement* "}"

Exp         = Exp1 "?" Exp1 ":" Exp                     --conditional
            | Exp1
Exp1        = Exp1 "??" Exp2                            --unwrapelse
            | Exp2
Exp2        = Exp3 ("||" Exp3)+                         --or
            | Exp3 ("&&" Exp3)+                         --and
            | Exp3
Exp3        = Exp4 ("|" Exp4)+                          --bitor
            | Exp4 ("^" Exp4)+                          --bitxor
            | Exp4 ("&" Exp4)+                          --bitand
            | Exp4
Exp4        = Exp5 ("<="|"<"|"=="|"!="|">="|">") Exp5   --compare
            | Exp5
Exp5        = Exp5 ("<<" | ">>") Exp6                   --shift
            | Exp6
```

```
Exp6        = Exp6 ("+" | "-") Exp7                          --add
            | Exp7
Exp7        = Exp7 ("*"| "/" | "%") Exp8                     --multiply
            | Exp8
Exp8        = Exp9 "**" Exp8                                 --power
            | Exp9
            | ("#" | "-" | "!" | some | random) Exp9         --unary
Exp9        = true ~mut
            | false ~mut
            | floatlit ~mut
            | intlit ~mut
            | no Type ~mut                                   --emptyopt
            | Exp9 ("(" | "?(") ListOf<Exp, ","> ")" ~mut    --call
            | Exp9 ("[" | "?[") Exp "]"                      --subscript
            | Exp9 ("." | "?.") id                           --member
            | stringlit ~mut
            | id                                             --id
            | Type_array "(" ")" ~mut                        --emptyarray
            | "[" NonemptyListOf<Exp, ","> "]" ~mut          --arrayexp
            | "(" Exp ")" ~mut                               --parens

intlit      = digit+
floatlit    = digit+ "." digit+ (("E" | "e") ("+" | "-")? digit+)?
stringlit   = "\"" char* "\""
char        = ~control ~"\\" ~"\"" any
            | "\\" ("n" | "t" | "\"" | "\\")                 --escape
            | "\\u{" hex hex? hex? hex? hex? hex? "}"         --codepoint
control     = "\x00".."\x1f" | "\x80".."\x9f"
hex         = hexDigit
mut         = ~"==" "=" | "++" | "--"

let         = "let" ~alnum
const       = "const" ~alnum
struct      = "struct" ~alnum
function    = "function" ~alnum
if          = "if" ~alnum
else        = "else" ~alnum
while       = "while" ~alnum
repeat      = "repeat" ~alnum
```

```
  for         = "for" ~alnum
  in          = "in" ~alnum
  random      = "random" ~alnum
  break       = "break" ~alnum
  return      = "return" ~alnum
  some        = "some" ~alnum
  no          = "no" ~alnum
  true        = "true" ~alnum
  false       = "false" ~alnum
  keyword     = let | const | struct | function | if | else | while | repeat
              | for | in | break | return | some | no | random | true | false
  id          = ~keyword letter alnum*

  space      += "//" (~"\n" any)*                              --comment
}
```

4 Standard Library

The following identifiers are pre-defined in a scope that surrounds the program. This means that none of these identifiers may be declared anywhere in a program (but they may be used as fields, though doing so is not recommended).

- `function print(s: any): void`

  Writes a representation of s to standard output.

- `function codepoints(s: string): [int]`

  Returns the code points of the s.

- `function bytes(s: string): [int]`

  Returns the bytes of the UTF-8 encoding of s.

- `const π`

  Returns the best approximate value of π in the type `float`.

- `function sqrt(x: float): float`

Returns the square root of x.

- `function sin(x: float): float`

  Returns the sine of x radians.

- `function cos(x: float): float`

  Returns the cosine of x radians.

- `function exp(x: float): float`

  Returns e^x.

- `function ln(x: float): float`

  Returns the natural log of x.

- `function hypot(x: float, y: float): float`

  Returns the hypotenuse of a right triangle with sides $|x|$ and $|y|$.